

HashMap Implementation: Complete Dry Run

Let's do a complete dry run of this implementation with diagrams. I'll use a small hash table size (SIZE = 5) instead of 1009 so collisions happen and we can see how the linked lists work. The logic is exactly the same.

Initial State

```
MyHashMap mp;
```

Constructor runs

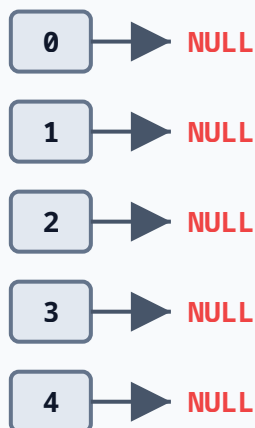
```
table.resize(SIZE, nullptr);
```

Suppose

SIZE = 5

Initially

Index



Operation 1

```
put(1,100);
```

Step 1

Compute hash

```
idx = 1 % 5 = 1;
```

Step 2

```
curr = table[1];
```

Currently

```
curr = NULL
```

So while loop doesn't execute.

Step 3

Create new node

```
Node* node = new Node(1,100);
```

Memory

```
(key=1,value=100,next=NULL)
```

Step 4

```
node->next = table[1];
```

Since

```
table[1]=NULL
```

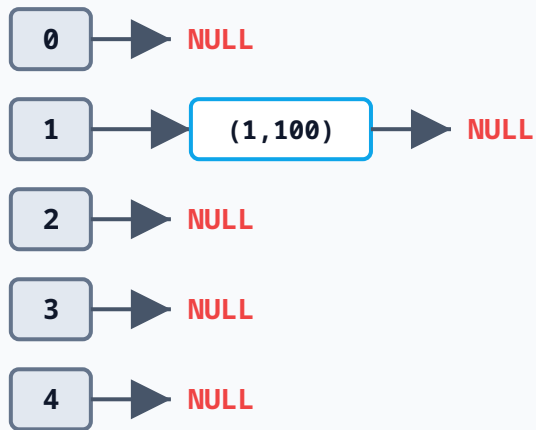
it becomes



Step 5

```
table[1]=node;
```

Table becomes



Operation 2

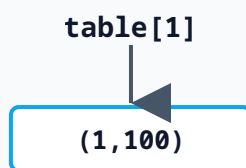
```
put(6, 200);
```

Hash

```
6%5=1
```

Collision!!

Current bucket



Traverse

```
curr=table[1];
```

Current node

```
(1, 100)
```

Compare

```
curr->key==6 ?
```

No

Move

```
curr=curr->next;
```

Now

```
curr=NULL
```

Loop ends.

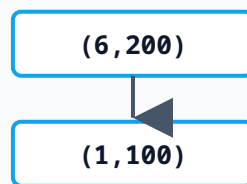
Create node

```
(6,200)
```

Insert at head

```
node->next=table[1];
```

Means

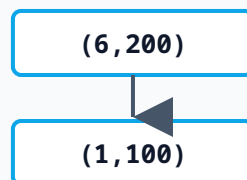


Then

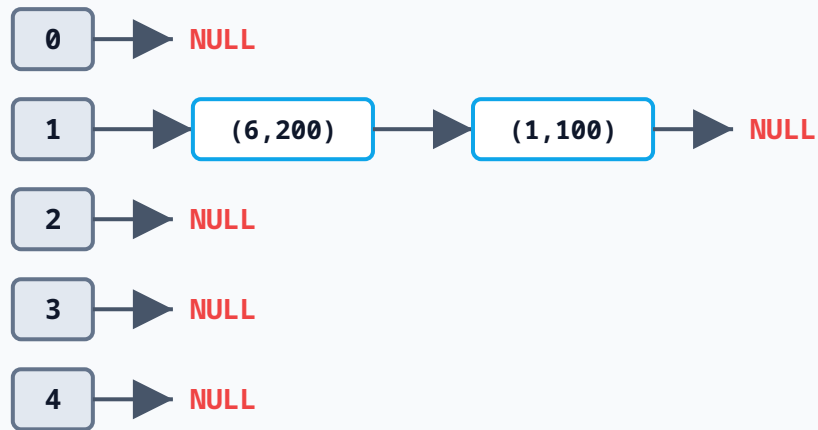
```
table[1]=node;
```

Final

Bucket 1



Whole table



Notice how both keys exist, even though they hash to the same bucket.

Operation 3

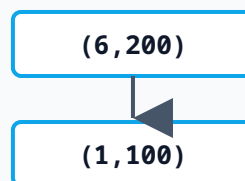
```
put(11, 300);
```

Hash

$$11 \% 5 = 1$$

Again bucket 1.

Current



Traverse

First

$$6 \neq 11$$

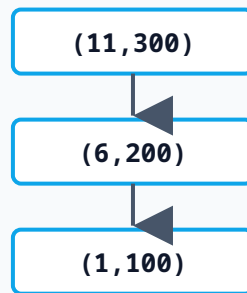
Move

$$1 \neq 11$$

Move

NULL

Insert at head



Table

1



Operation 4

```
get(6);
```

Hash

$6\%5=1$

Current bucket



First node

11

Compare

11==6 ?

No

Move

6

Compare

6==6

Yes

Return

200

Operation 5

```
put(6,999);
```

Hash

6%5=1

Bucket



Traverse

`11!=6`

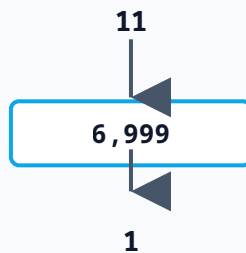
Move

`6==6`

Found

`curr->value=999;return;`

Table becomes



No new node created.

Operation 6

```
remove(6);
```

Hash

`6%5=1`

Current bucket



Initially

curr=11

prev=NULL

Iteration 1

curr=11

prev=NULL

Not found

Update

prev=11

curr=6

Iteration 2

Now

prev=11

curr=6

Found

Execute

prev->next=curr->next;

Meaning

Before



After



Then

```
delete curr;
```

Memory of node

6

is freed.

Final table



Operation 7

```
remove(11);
```

Hash

$11\%5=1$

Bucket



Initially

```
prev=NULL
```

```
curr=11
```

Found immediately.

Execute

```
table[idx]=curr->next;
```

Meaning

Instead of

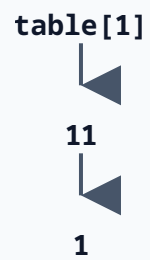


Table becomes



Delete

```
11
```

Final

Bucket1

1

Operation 8

```
get(6);
```

Hash

6%5=1

Bucket

1

Compare

1!=6

Move

NULL

Return

-1

Why insert at the head?

Suppose bucket contains



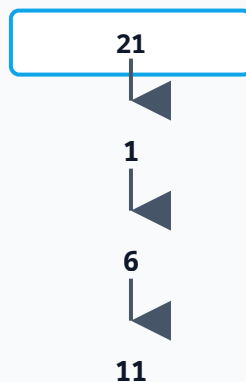
New node

21

Instead of traversing to the end, we do

```
node->next = table[idx];  
table[idx] = node;
```

Result



This takes $O(1)$ time.

Complete Flow

`put(1,100)`

Bucket1

1



`put(6,200)`

Bucket1

6
1



`put(11,300)`

Bucket1

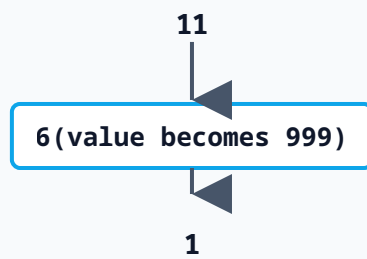
11
6
1



`get(6)`



`put(6,999)`



`remove(6)`



`remove(11)`

1



`get(6)`

Not Found

Return -1

The key idea to remember

A hash map is not one linked list. It is:

- An array of buckets (`vector<Node*> table`), where each element is the head of a linked list.
- The hash function (`key % SIZE`) decides which linked list (bucket) to use.
- Each linked list stores all key-value pairs that collide into the same bucket (this is called separate chaining).

put, get, and remove only traverse one bucket's linked list, not the entire table. This is why the average time complexity is $O(1)$ when the hash function distributes keys well.